



CS-302 Artificial Intelligence

WEEK 05

DR. SAMIA IJAZ

Organization of Lecture

1. Informed Search Strategies

1. Introduction
2. Heuristic Search
3. Heuristic Function
4. Best First Search
5. Beam Search
6. A* (A-Star) Algorithm
7. AO* Algorithm
8. Hill Climbing
9. Branch and Bound
10. State Space Search

1. Introduction (1/4)

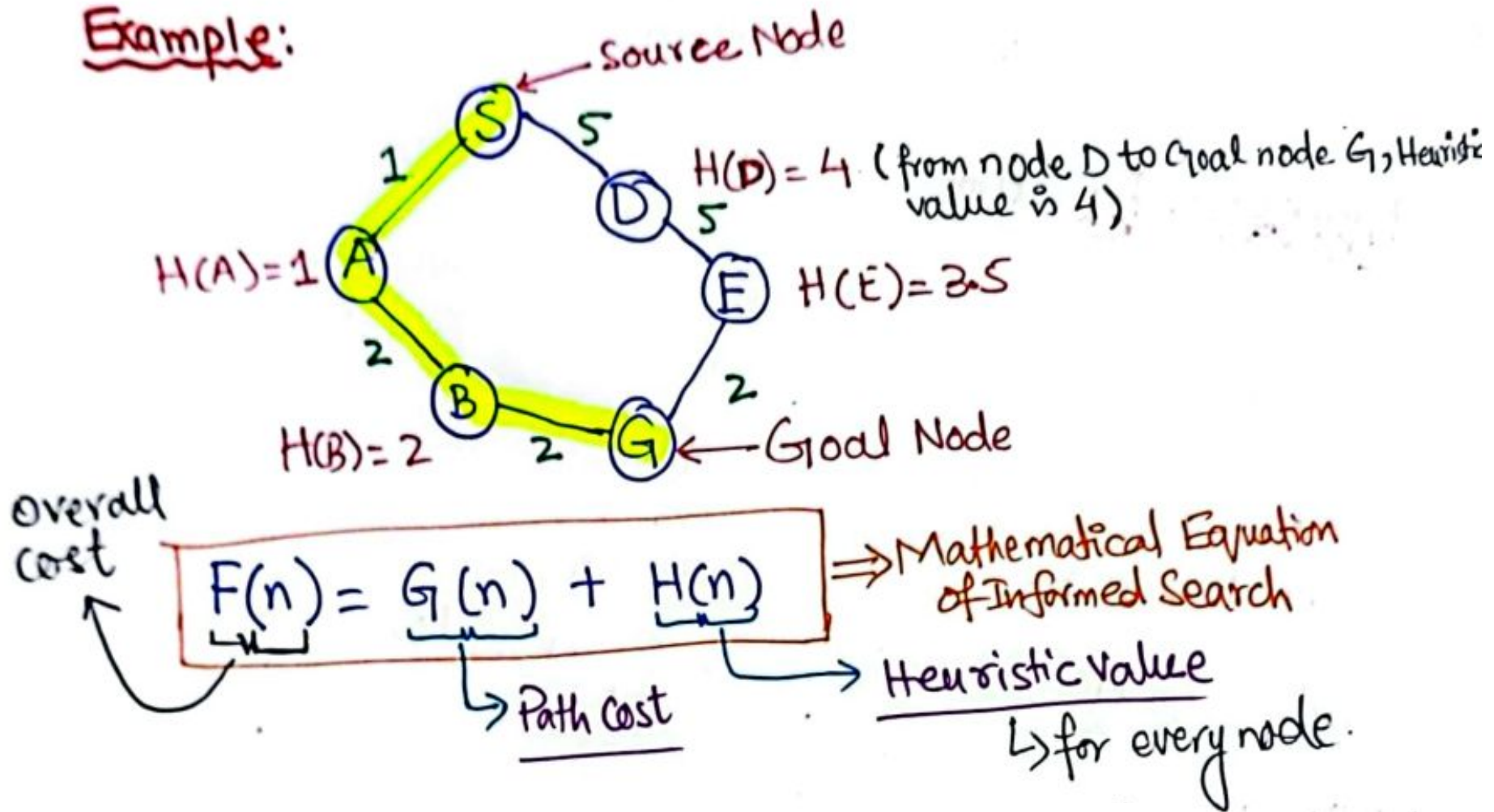
- Informed search (also called heuristic search) uses:
 - extra knowledge (heuristics) about the problem to guide the search more intelligently.
- Unlike uninformed search (e.g., BFS, DFS), which blindly explores the graph,
 - informed search uses heuristics to prioritize nodes.

1. Introduction (2/4)

- A heuristic is a rule of thumb, an educated guess, or a shortcut that helps:
 - an algorithm decide which path to explore first.
- In search problems, it's a function:
 - $h(n)$ that estimates the cost from a node n to the goal.
 - It does not guarantee correctness, but helps make decisions faster.
- For example: “Which direction looks more promising based on what I know?”

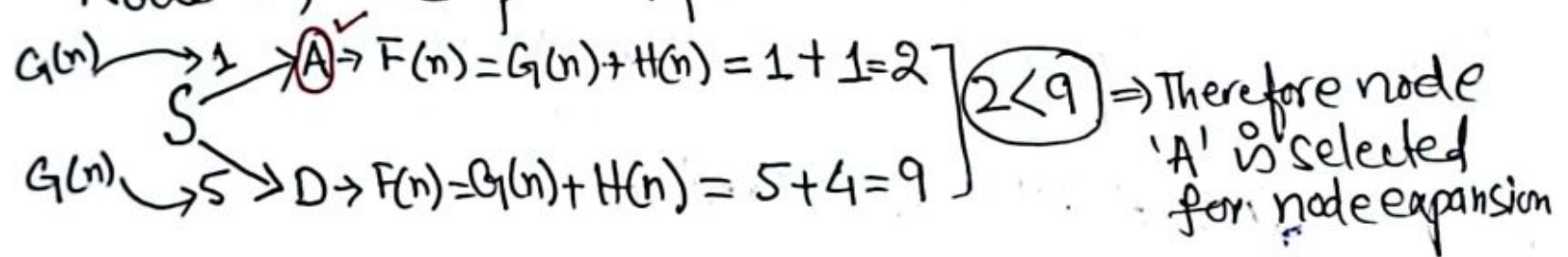
1. Introduction (3/4)

Example:



1. Introduction (4/4)

→ let us say we want to go to Goal Node 'G' from Source Node 'S'. The optimal path will be:



$$A \rightarrow B \left[F(n) = \overbrace{G(n)}^{1+2} + H(n) = (1+2) + 2 = 5 \right]$$

$B \rightarrow G$; Reached to Goal Node.

Total Path Cost (TPC)

$$TPC = S \xrightarrow{1} A \xrightarrow{2} B \xrightarrow{2} G$$

$$TPC = 1 + 2 + 2 = 5$$

$$\boxed{TPC = 5}$$

2. Heuristic Search

2) Heuristic Search:

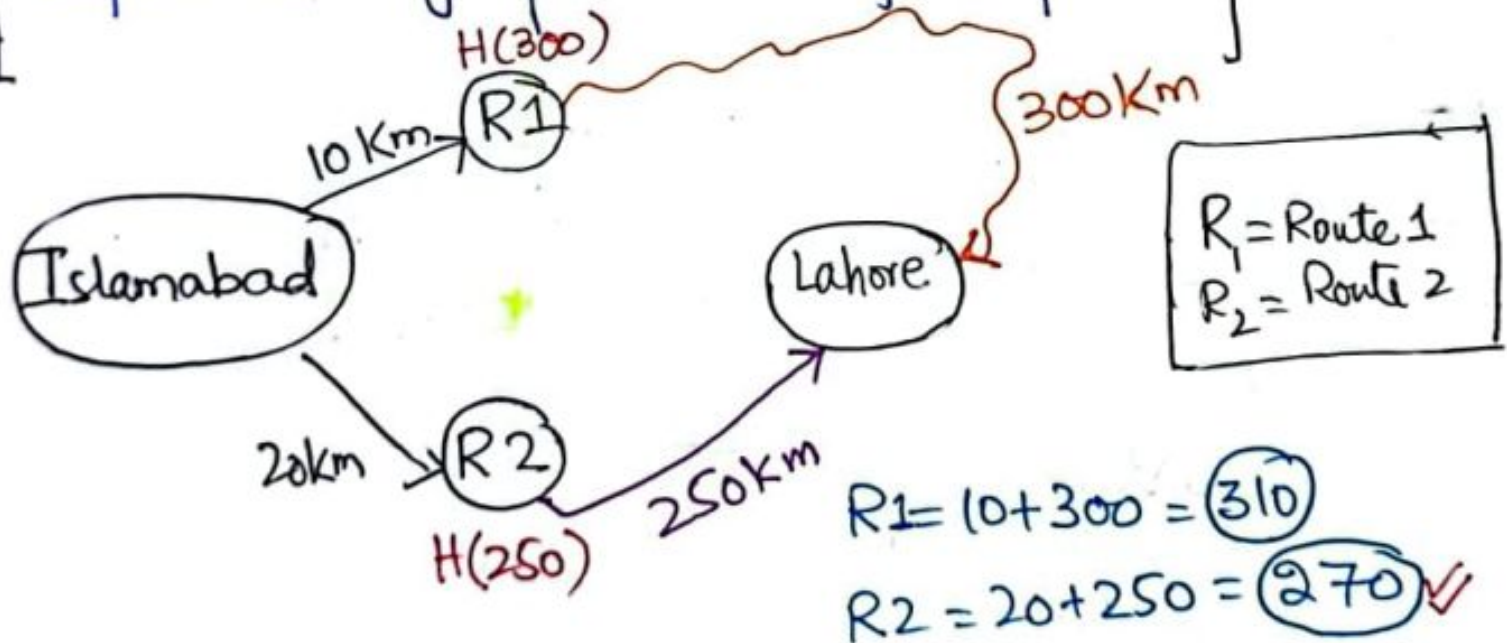
- ↳ Tries to optimize a problem using heuristic function.
- ↳ Optimization means trying to solve a problem in minimum cost / minimum number of steps.

3. Heuristic Function (1/5)

3) Heuristic Function:

It is a function ' $H(n)$ ' that gives an estimation on the cost of getting from node ' n ' to the GOAL state. Estimated Value

Helps in selecting optimal node for expansion



3. Heuristic Function (3/5)

Types of Heuristic Search

↳ (i) Admissible: In this Heuristic function, never overestimate the cost of reaching the Goal node.

* $h(n)$ is always less than or equal to actual cost of lowest-cost path from node 'n' to goal

$$H(n) \leq H'(n)$$

$H(n) = \text{Estimated Cost}$ $H'(n) = \text{Actual Cost}$
--

(ii) Non-Admissible: Overestimate $\Rightarrow H(n) > H'(n)$

3. Heuristic Function (4/5)

Example

$$\rightarrow H(B)=3, H(C)=4, H(D)=5$$

$$\rightarrow F(n) = H(n) + G(n)$$

$$\rightarrow \underline{B} \Rightarrow F(n) = 1+3=4 \checkmark$$

$$\rightarrow \underline{C} \Rightarrow F(n) = 1+4=5$$

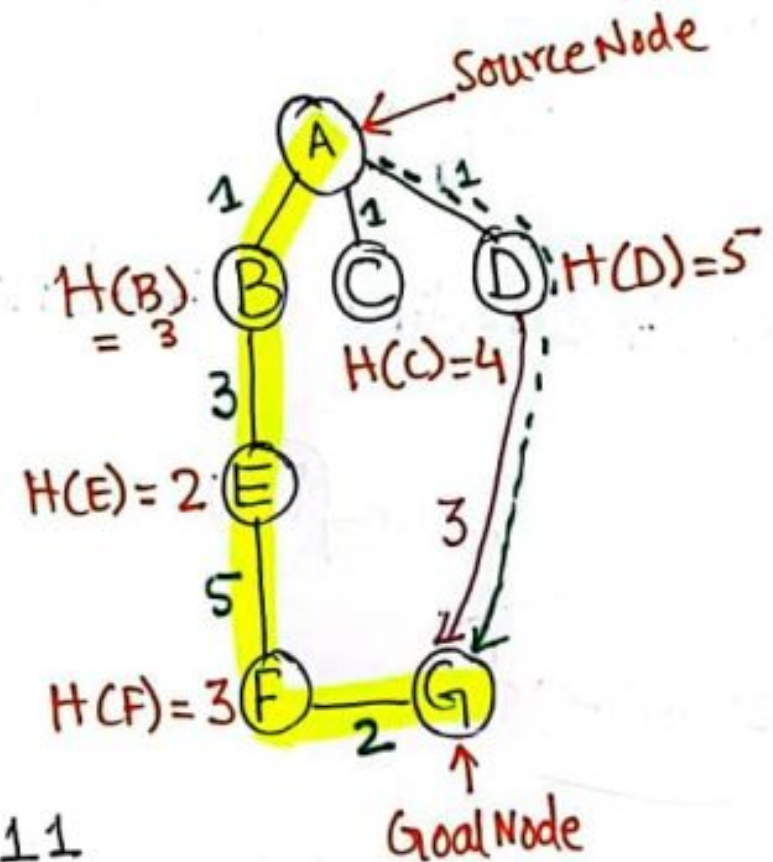
$$\rightarrow \underline{D} \Rightarrow F(n) = 1+5=6$$

B:-

$$\rightarrow \text{Actual Cost} = H'(B) = 1+3+5+2 = 11$$

$$4 \quad H(B)=3$$

$$\rightarrow H(B) < H(B') \Rightarrow 3 < 11 \Rightarrow \text{Admissible}$$



3. Heuristic Function (5/5)

→ In Case of E $\Rightarrow H(E) < H'(E) \Rightarrow 2 < 11$ (Admissible)

→ In case of F $\Rightarrow H(F) < H'(F) \Rightarrow 3 < 11$ (Admissible)

→ In Admissible Search, at each node $H(n) \leq H'(n)$

→ In Non-Admissible Search, If we had selected path $A \xrightarrow{1} D \xrightarrow{3} G$, we would have come up with $H(D) > H'(D) \Rightarrow 5 > 1+3 \Rightarrow 5 > 4$

4. Best First Search (1/6)

- Heuristic / Informed search
- Uses evaluation function to decide which adjacent node is most promising
- Priority queue is used

4. Best First Search (2/6)

Algorithm:

Priority Queue 'PQ' containing initial states → Sorted Order

Loop

if PQ = Empty Return FAIL

Else

NODE ← Remove-First(PQ)

if NODE = GOAL

Return Path from Initial to NODE

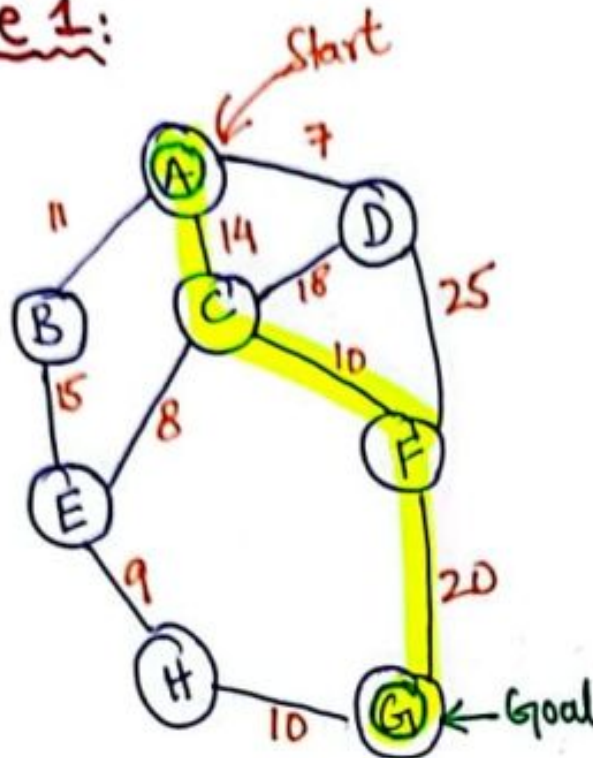
Else

Generate all Successors of NODE
and insert newly generate NODE
into PQ according to cost value.

END LOOP

4. Best First Search (3/6)

Example 1:



→ Usually given with Question (6)
Straight line distance

$$A \rightarrow G_1 = 40$$

$$B \rightarrow G_1 = 32$$

$$C \rightarrow G_1 = 25$$

$$D \rightarrow G_1 = 35$$

$$E \rightarrow G_1 = 19$$

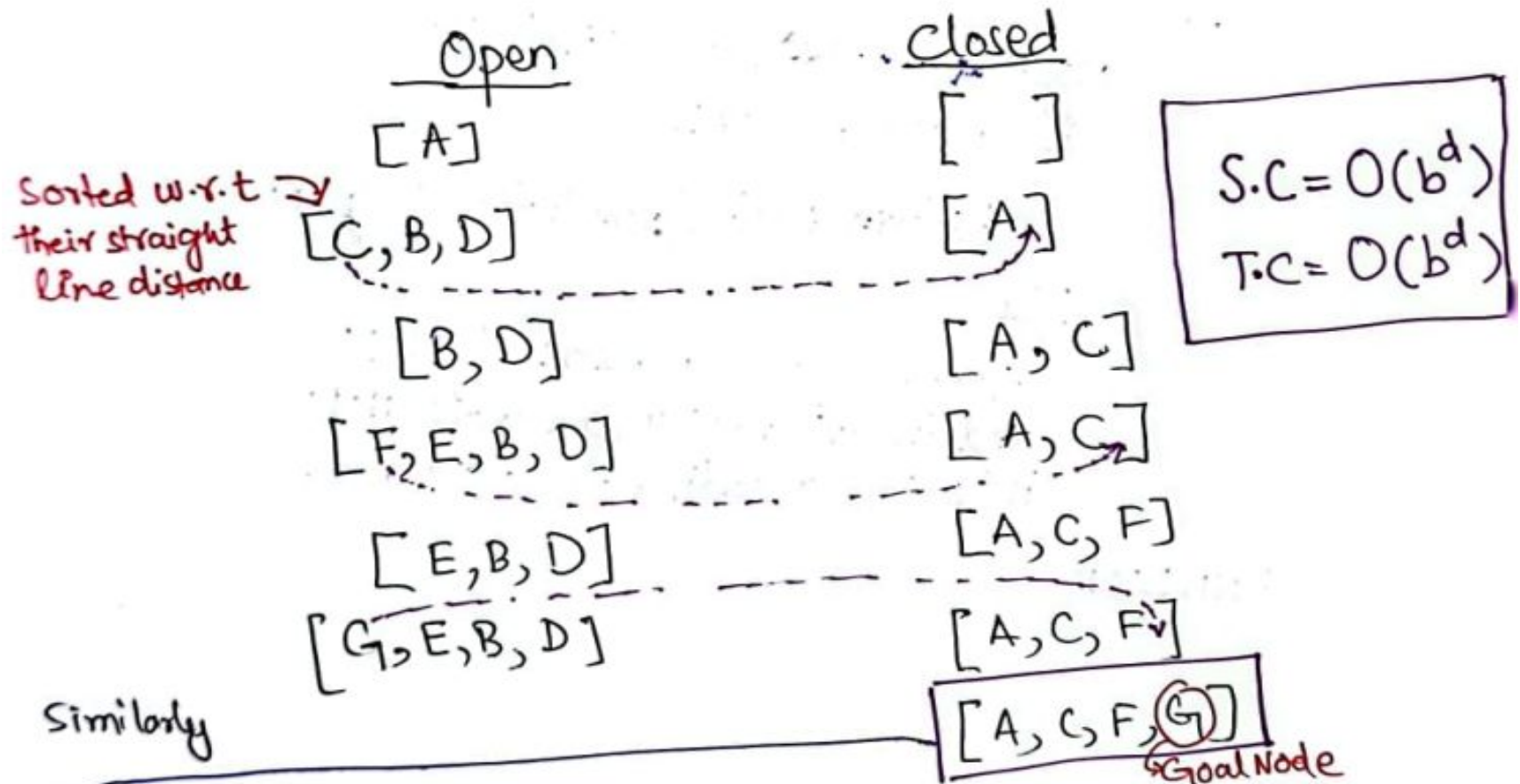
$$F \rightarrow G_1 = 17$$

$$G \rightarrow G_1 = 0$$

$$H \rightarrow G_1 = 10$$

→ We need to maintain two lists i.e., open List & Closed List.

4. Best First Search (4/6)



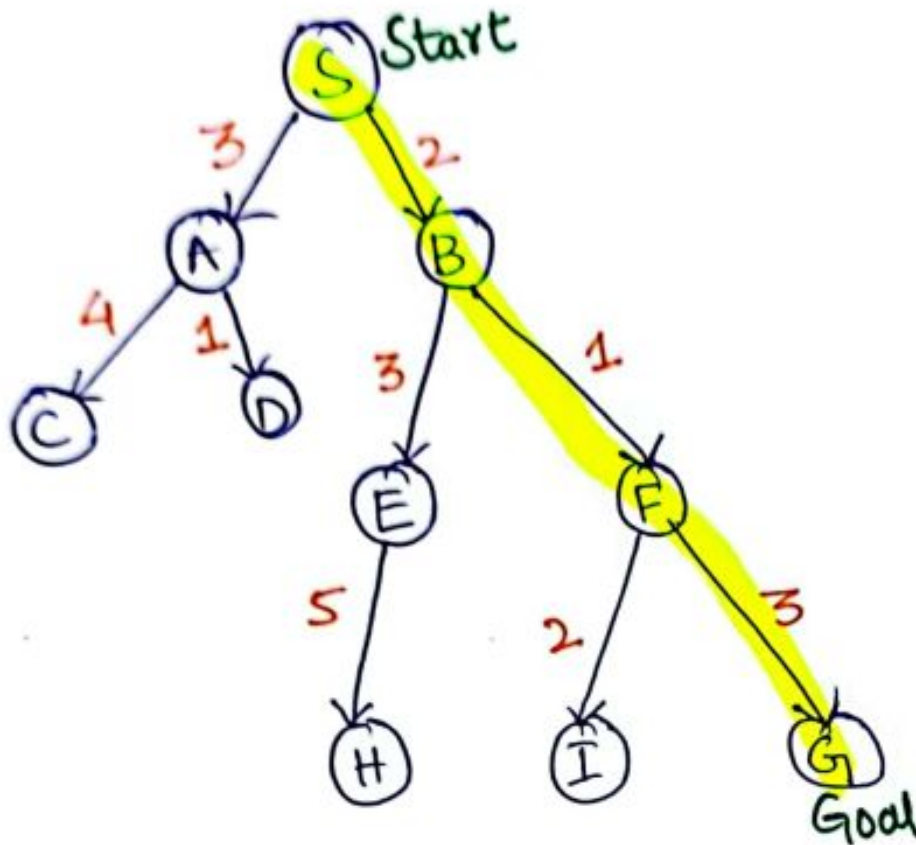
$$S.C = O(b^d)$$

$$T.C = O(b^d)$$

- Return this path i.e, $A \xrightarrow{14} C \xrightarrow{10} F \xrightarrow{20} G = 14 + 10 + 20 = 44$
- It might be possible that some other optimal path exists, so best first search will not always give you the best path

4. Best First Search (5/6)

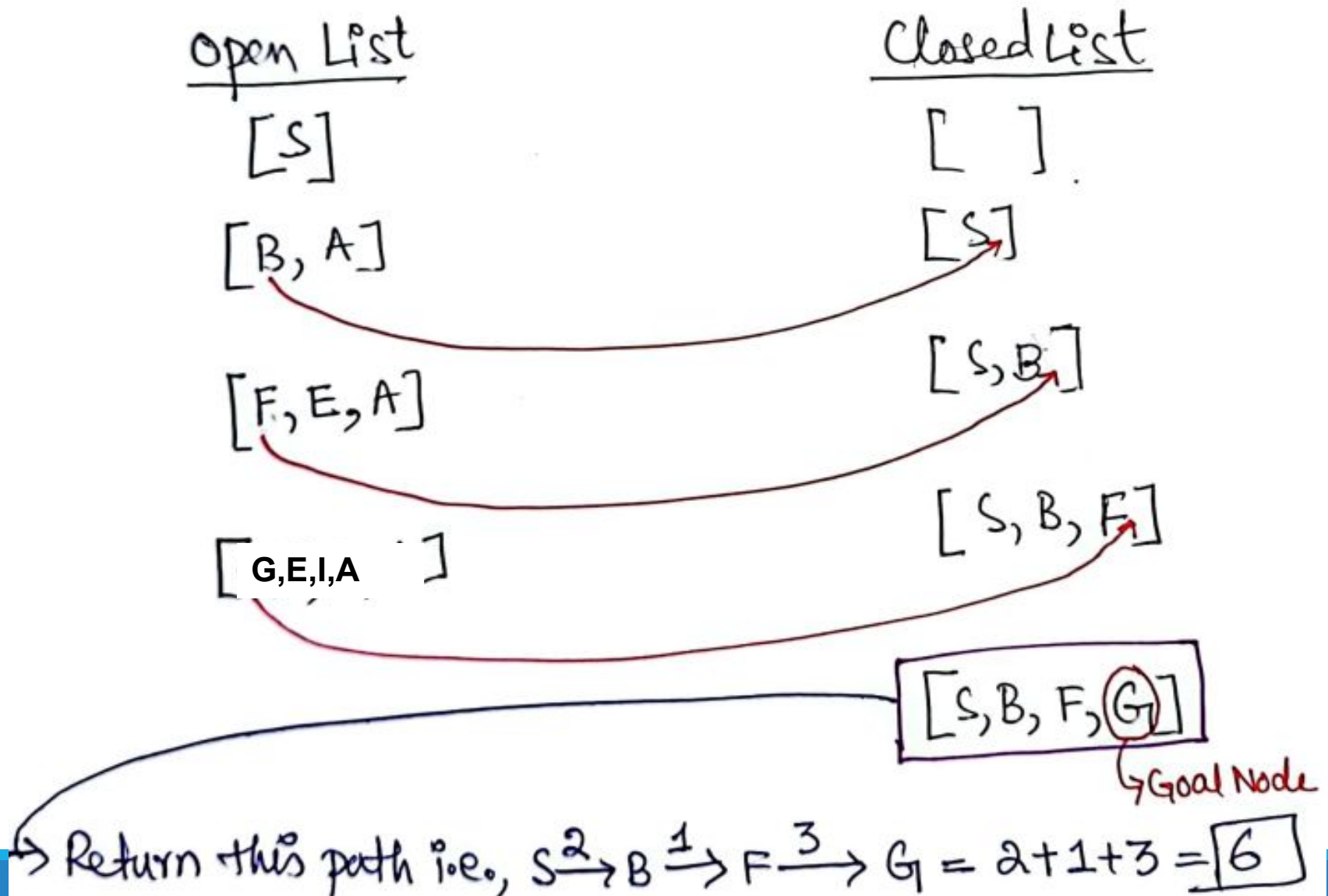
Example 2:



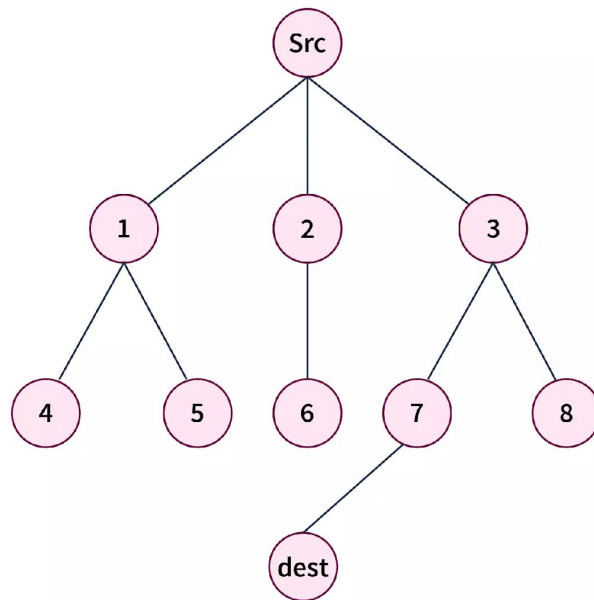
Given:

<u>node(n)</u>	<u>H(n)</u>
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

4. Best First Search (6/6)

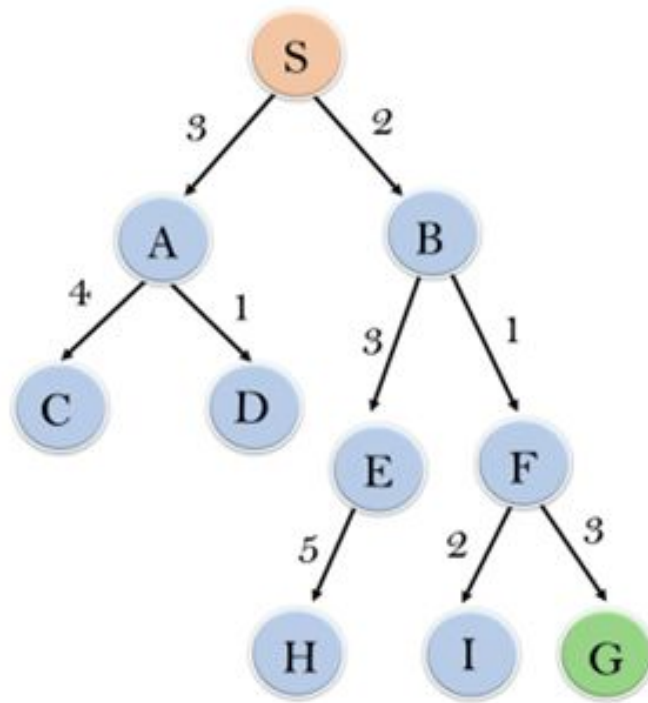


4. Best First Search – Activity Question



Src	20
1	22
2	21
3	10
4	25
5	24
6	30
7	5
8	12
dest	0

4. Best First Search – Activity Question



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

5. Beam Search (1/4)

1) Beam Search

optimized version of Best First Search.

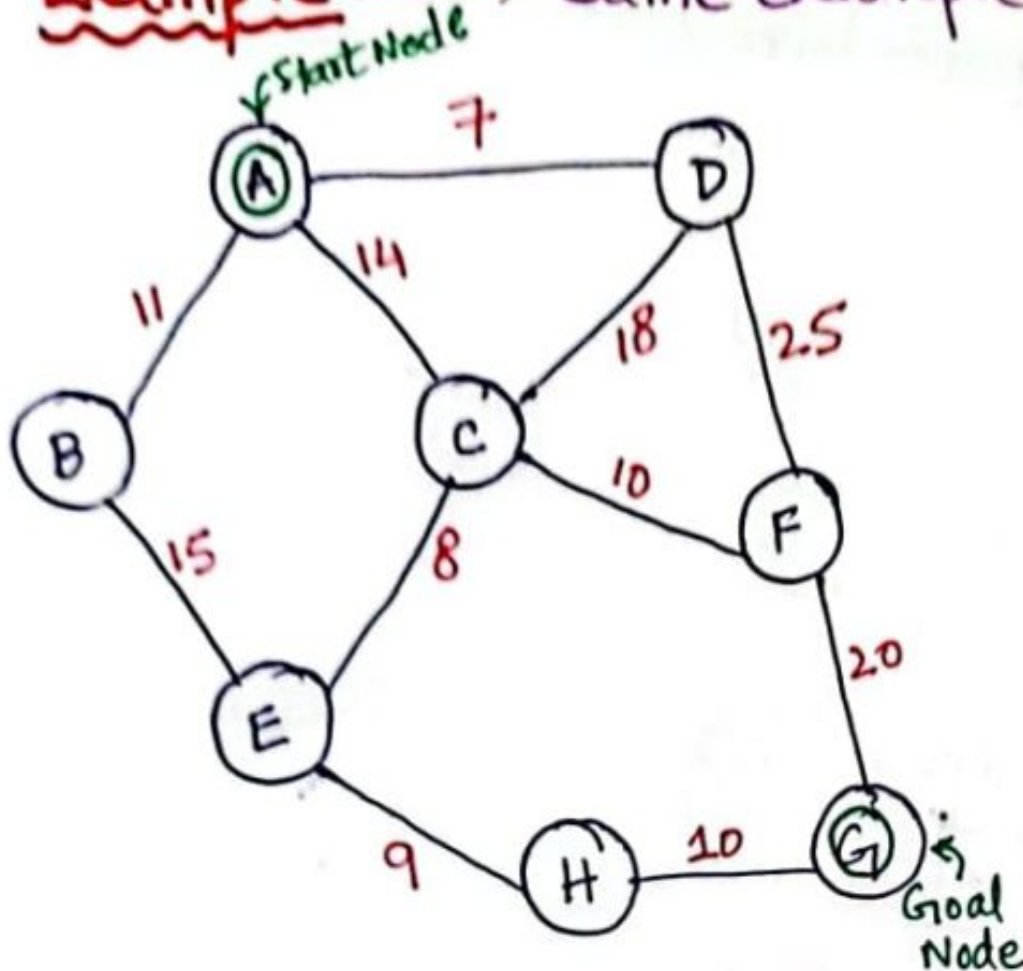
- ↳ Heuristic Search Algorithm.
- ↳ Explores a Graph by expanding the most promising node in a limited set.
- ↳ Reduces Memory Requirement. (Limited/Less no. of selected candidates will result in less memory requirement).
- ↳ Greedy Algorithm $\Rightarrow$ An algorithm for solving a problem by selecting the best option available at the moment. It doesn't worry whether the current best result will bring the overall optimal result.

5. Beam Search (2/4)

- ↳ Predetermined no. of best partial solutions are kept as candidates.
- ↳ Predetermined no. is also called Beam Value (β).

5. Beam Search (3/4)

Example $\Rightarrow$ Same example as in Best First Search.

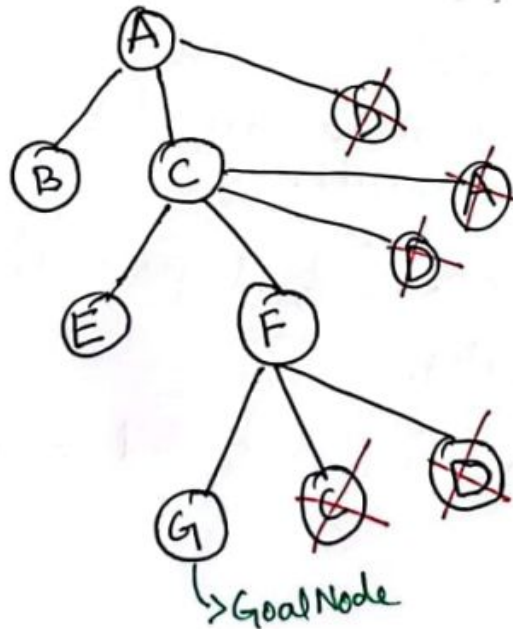


$A \rightarrow G = 40$
 $B \rightarrow G = 32$
 $C \rightarrow G = 25$
 $D \rightarrow G = 35$
 $E \rightarrow G = 19$
 $F \rightarrow G = 17$
 $H \rightarrow G = 10$
 $G \rightarrow G = 0$

5. Beam Search (4/4)

Solution

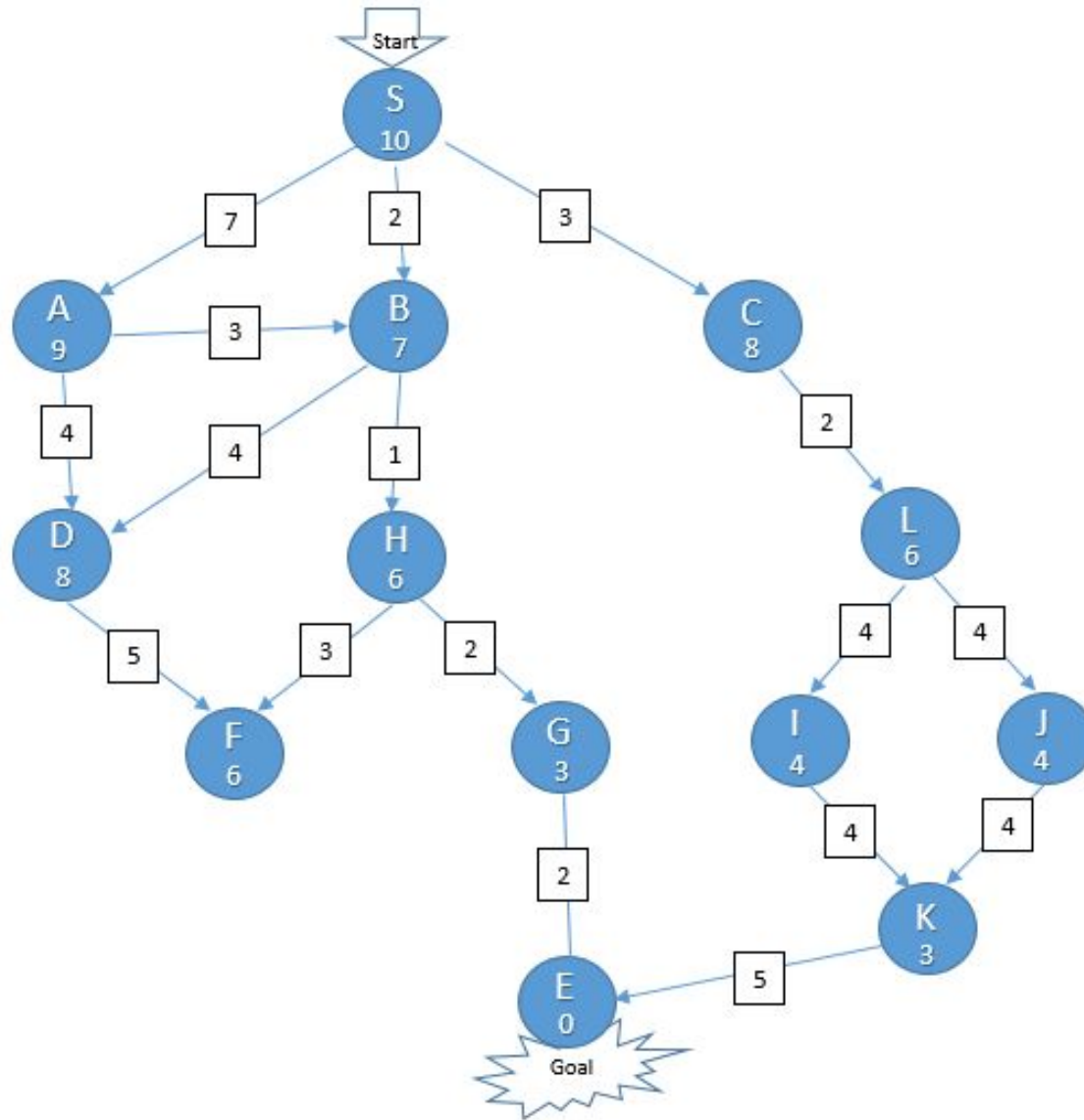
Let us value of ' β ' is given as 2 i.e., $\beta = 2$ (Best two nodes will be considered)



- ↳ We may receive the same solution as BFS but we achieve less memory requirement with the help of beam value (β).
- ↳ If we set $\beta = 1$, then it will become Hill Climbing Algorithm.

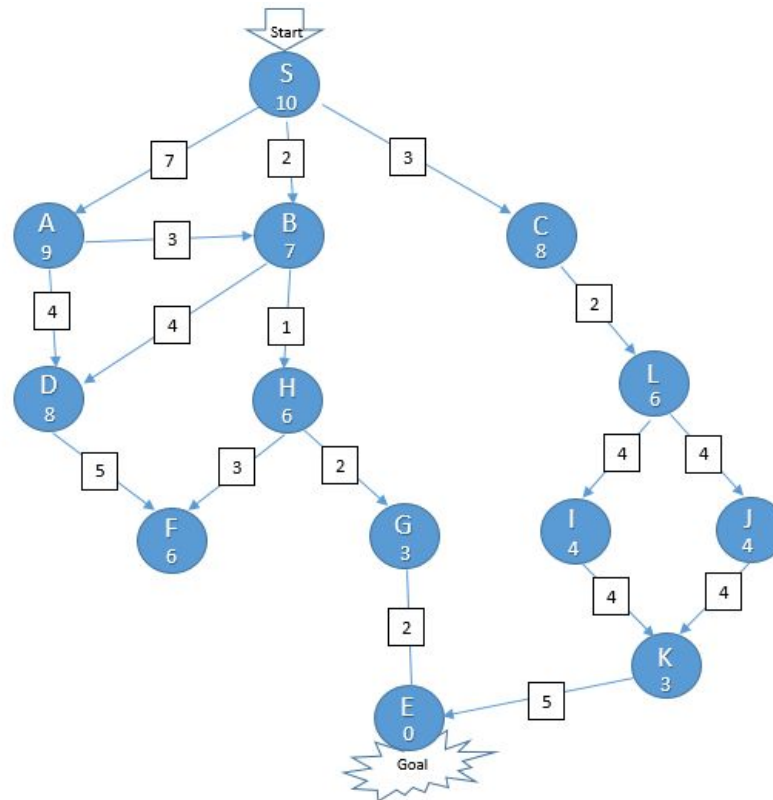
Bea

A bear



5. Beam Search – Activity Question

A beam width of $\beta = 2$



6. A* (A-Star) Algorithm (1/8)

2) A* Algorithm

↳ Uses heuristic function 'h(n)' and cost to reach the node 'n' from start state 'g(n)'.

↳ Finds shortest path through search space.

↳ Fast and optimal result

$$\underbrace{f(n)}_{\text{Total Estimated Cost}} = \underbrace{g(n)}_{\text{Cost to reach node}} + \overbrace{h(n)}^{\text{heuristic value (child node)}}$$

6. A* (A-Star) Algorithm (2/8)

Algorithm:

- ↳ i) Enter starting node in OPEN list.
- ii) If OPEN list is empty return FAIL.
- iii) Select node from OPEN list which has smallest value ($g+h$)
 - ↳ if node = Goal, return Success.
- iv) Expand node 'n' and generate all Successors.
 - ↳ compute ($g+h$) for each Successor node.
- v) If node 'n' is already in OPEN/ CLOSED, attach to back pointer.
- vi) Go to (i).

6. A* (A-Star) Algorithm (3/8)

Advantages:-

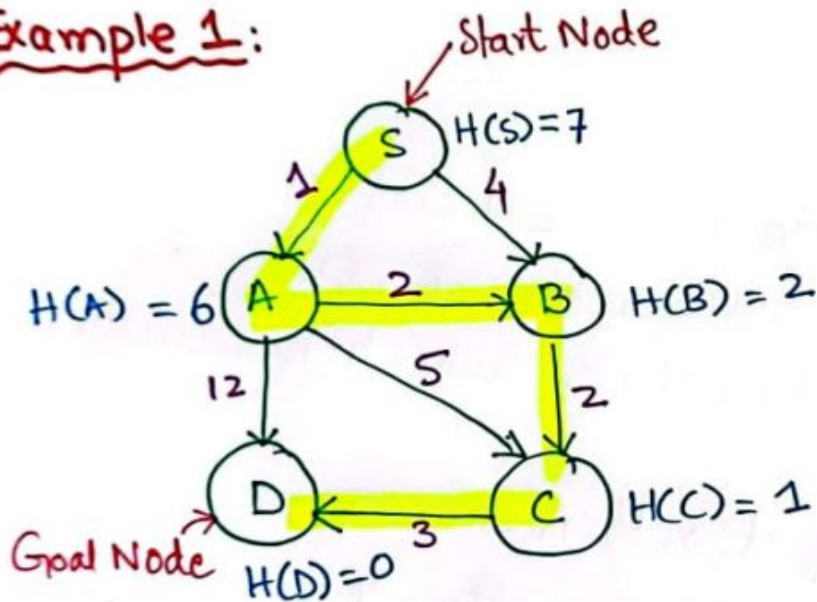
- i) Best Searching algorithm
- ii) Optimal and Complete
- iii) Solving complex problems.

Disadvantages:-

- i) complexity issue in its implementation.
- ii) Requires memory.

6. A* (A-Star) Algorithm (4/8)

Example 1:



$$S \rightarrow A = g(n) + h(n) = 1 + 6 = 7$$

$$S \rightarrow B = g(n) + h(n) = 2 + 4 = 6 \rightarrow \text{(Explored)}$$

$$S \rightarrow B \rightarrow C = (4 + 2) + 1 = 7 \rightarrow \text{(Explored)}$$

$\nwarrow$ "g" value from S $\rightarrow$ B $\nwarrow$ "g" value from B $\rightarrow$ C $\nwarrow$ h value of child 'C'

$$S \rightarrow B \rightarrow C \rightarrow D = (4 + 2 + 3) + 0 = 9$$

$\nwarrow$ "g" values $\nwarrow$ h value of child 'D'

R1

6. A* (A-Star) Algorithm (5/8)

Since $\rightarrow A$ is not explored yet. Let's explore it

$$S \rightarrow A \rightarrow B = (1+2) + 2 = 5 \checkmark \text{ explored}$$

$$S \rightarrow A \rightarrow C = (1+5) + 1 = 7$$

$$S \rightarrow A \rightarrow D = (1+12) + 0 = 13 \quad R2$$

Let's now explore $S \rightarrow A \rightarrow B = 5$ (small/less estimated cost)

$$S \rightarrow A \rightarrow B \rightarrow C = (1+2+2) + 1 = 6 \checkmark \text{ explored}$$

further explore $S \rightarrow A \rightarrow B \rightarrow C$ i.e.,

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D = (1+2+2+3) + 0 = 8 \quad R3$$

6. A* (A-Star) Algorithm (6/8)

Now only 1 route is left / yet to be explored i.e.,

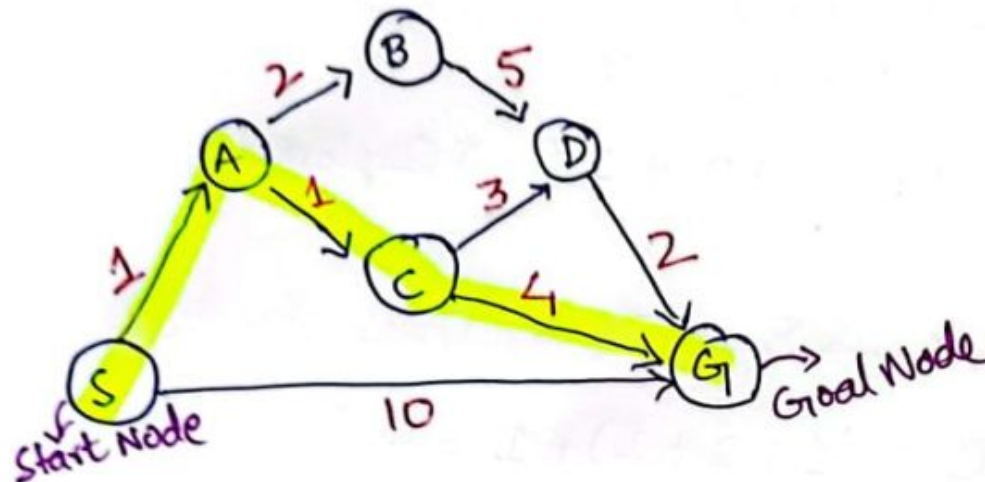
$$S \rightarrow A \rightarrow C \rightarrow D = (1+5+3)+0 = 9 \quad R4$$

Finally we have come up with 4 routes i.e. $R1(9)$, $R2(13)$, $R3(8)$ & $R4(9)$. We will select the route with minimum estimated cost i.e. $R3=8$ i.e.,

$$S \rightarrow A \rightarrow B \rightarrow C \rightarrow D = (1+2+2+3)+0 = 8$$

6. A* (A-Star) Algorithm (7/8)

Example 2



State	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

Solution

$$S \rightarrow A = g(A) + h(A) = 1 + 3 = 4$$

$$S \rightarrow G = g(G) + h(G) = 10 + 0 = 10 \rightarrow \text{Reached Goal Node. (Not further Explored)} \quad \times (R1)$$

$$S \rightarrow A \rightarrow B = \overbrace{(1+2)}^{\text{'g' values}} + \overbrace{4}^{\text{'h' value}} = 7 \checkmark$$

$$S \rightarrow A \rightarrow C = (1+1) + 2 = 4 \checkmark$$

6. A* (A-Star) Algorithm (8/8)

$$S \rightarrow A \rightarrow C \rightarrow D = (1+1+3)+6 = 11 \checkmark$$

$$S \rightarrow A \rightarrow C \rightarrow G = (1+1+4)+0 = 6 \rightarrow \text{Reached Goal Node, Not further Explored (R2)}$$

$$S \rightarrow A \rightarrow B \rightarrow D = (1+2+5)+6 = 14$$

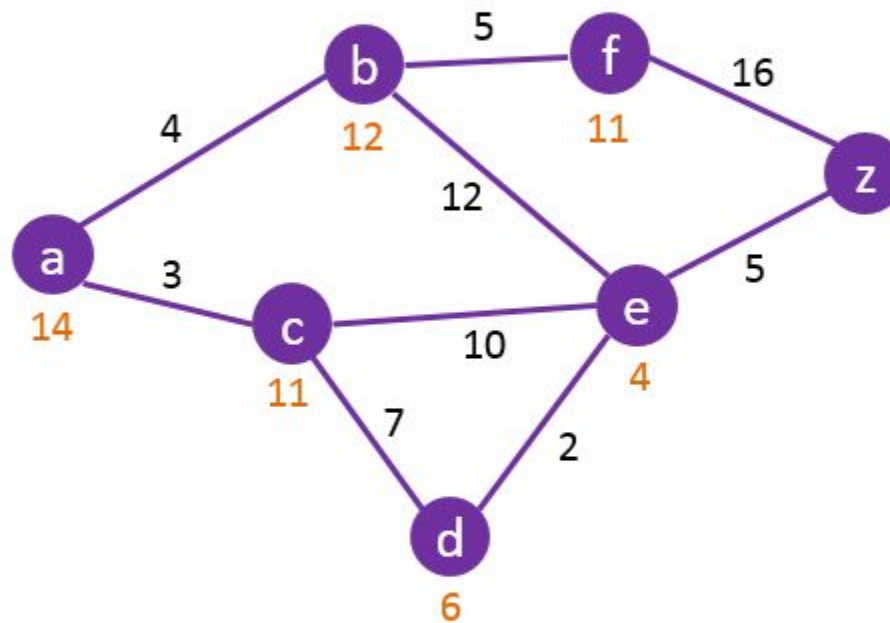
$$S \rightarrow A \rightarrow C \rightarrow D \rightarrow G = (1+1+3+2)+0 = 7 \rightarrow \text{Reached Goal Node (R3)}$$

$$S \rightarrow A \rightarrow B \rightarrow D \rightarrow G = (1+2+5+2)+0 = 10 \rightarrow \text{Reached Goal Node, (R4)}$$

The route with minimum cost will be selected i.e. R2

$$R2 = S \rightarrow A \rightarrow C \rightarrow G = 1+1+4 = 6$$

Class Activity



7. AO* (AND-OR-Star) Algorithm (1/4)

3) AO* Algorithm

AND-OR Graphs:-

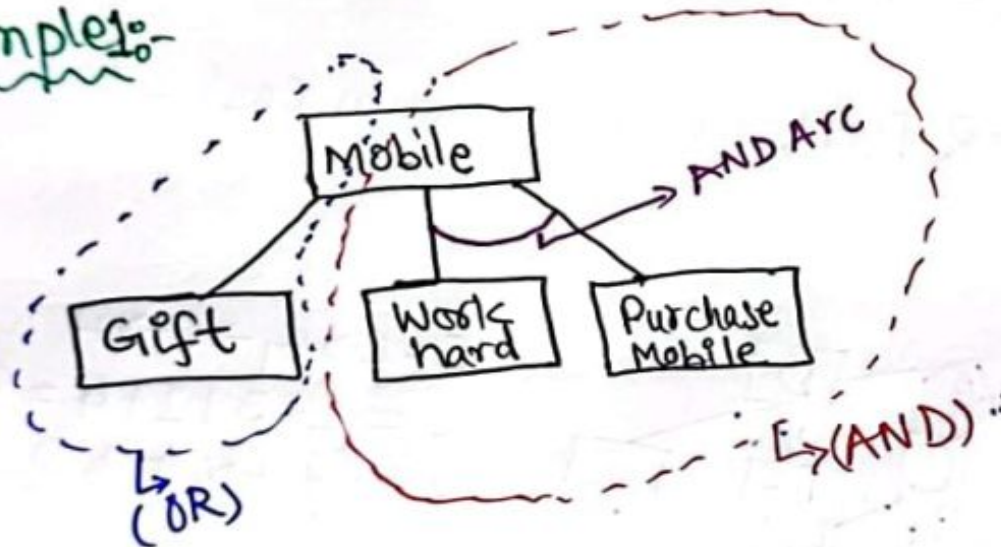
↳ Useful for representing the problem solution that can be solved by decomposing them into smaller set of problems all of which must be solved.

↳ Decomposing → AND ARCS → May point to 'n' no. of Successor nodes.

↳ Algo. like Best-Fit search can be used that has the ability to handle AND arcs.

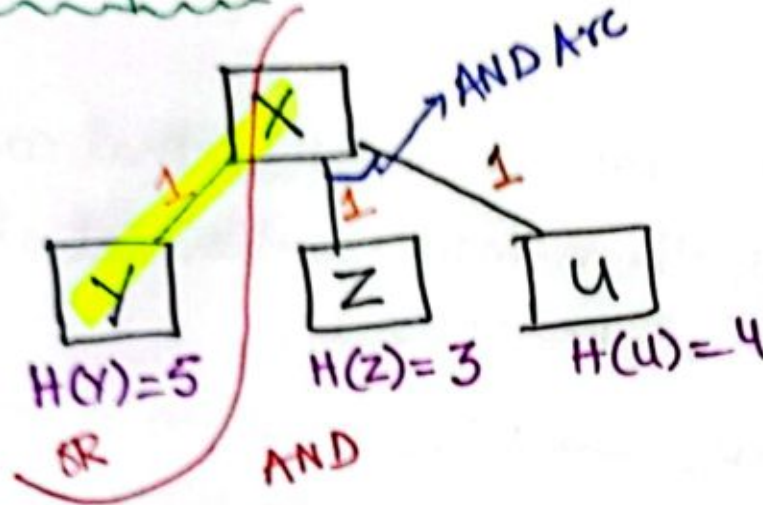
7. AO* (AND-OR-Star) Algorithm (2/4)

Example:-



7. AO* (AND-OR-Star) Algorithm (3/4)

Example 2:-



OR Part

$$X \rightarrow Y = 1 + 5 = 6 \checkmark$$

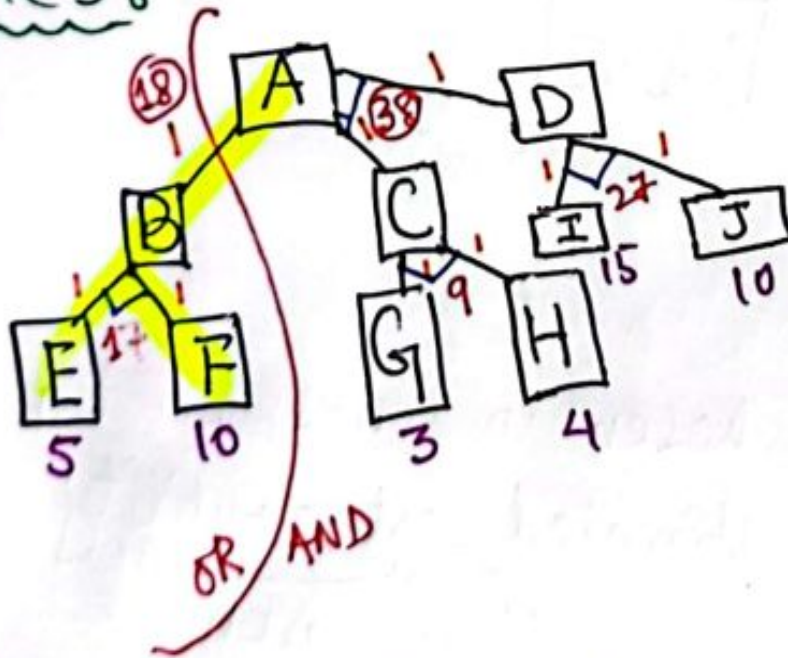
AND Part

$$X \rightarrow Z \rightarrow X \rightarrow U = 1 + 3 + 1 + 4 = 9$$

→ We will choose OR Part (with minimum cost = 6).

7. AO* (AND-OR-Star) Algorithm (4/4)

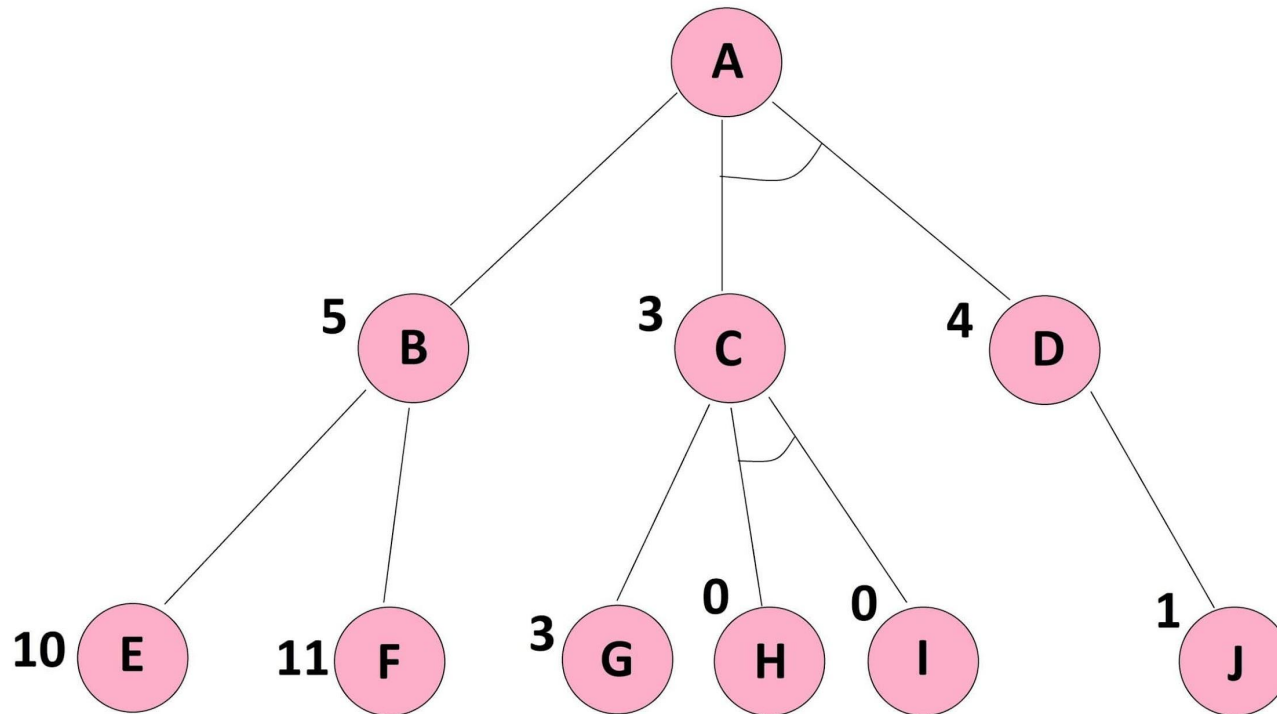
Example 3:-



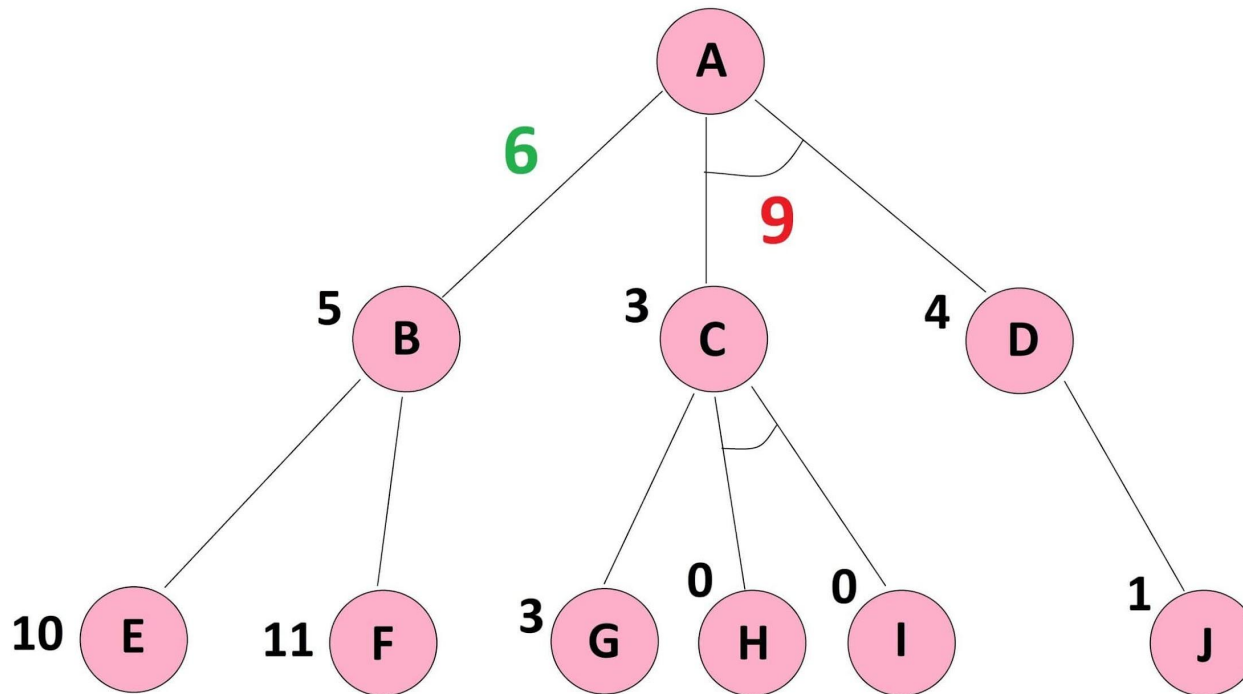
$C = C \rightarrow G \rightarrow C \rightarrow H$
 $C = 1 + 3 + 1 + 4 = 9$
Similarly for A & D.

↳ The OR part will be chosen (it has minimum cost i.e. 18)

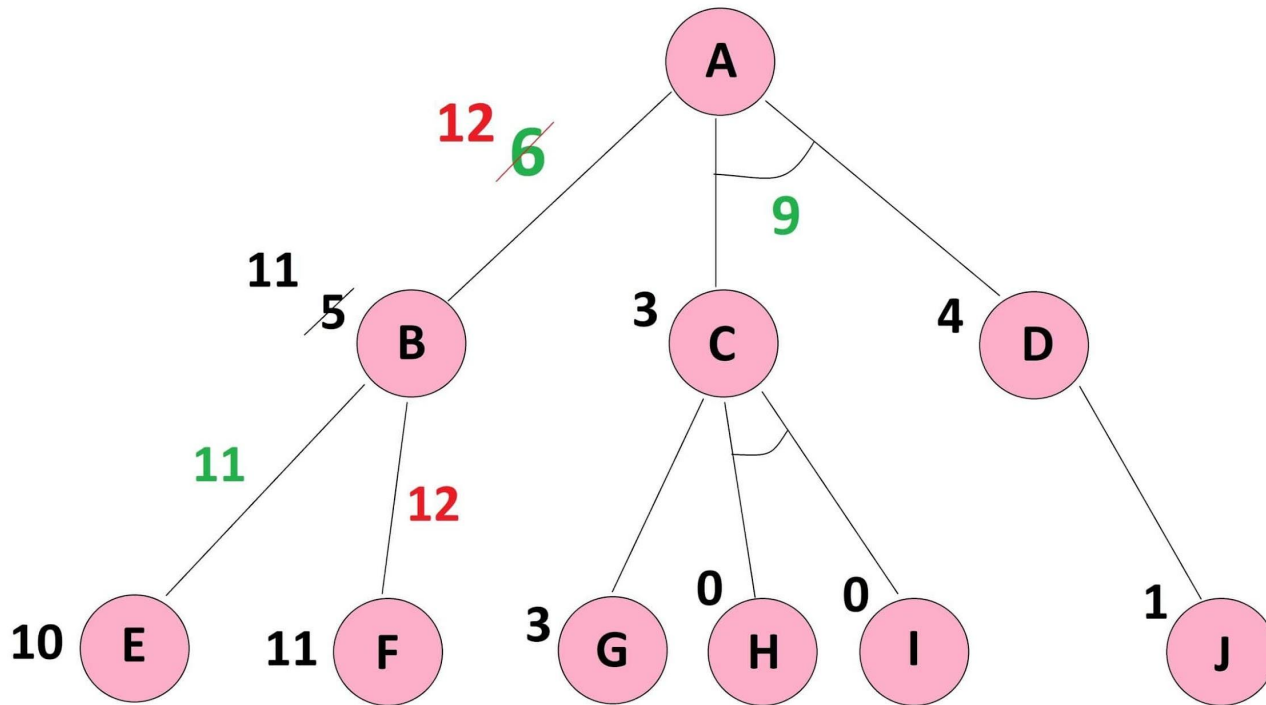
7. AO* (AND-OR-Star) Algorithm – example



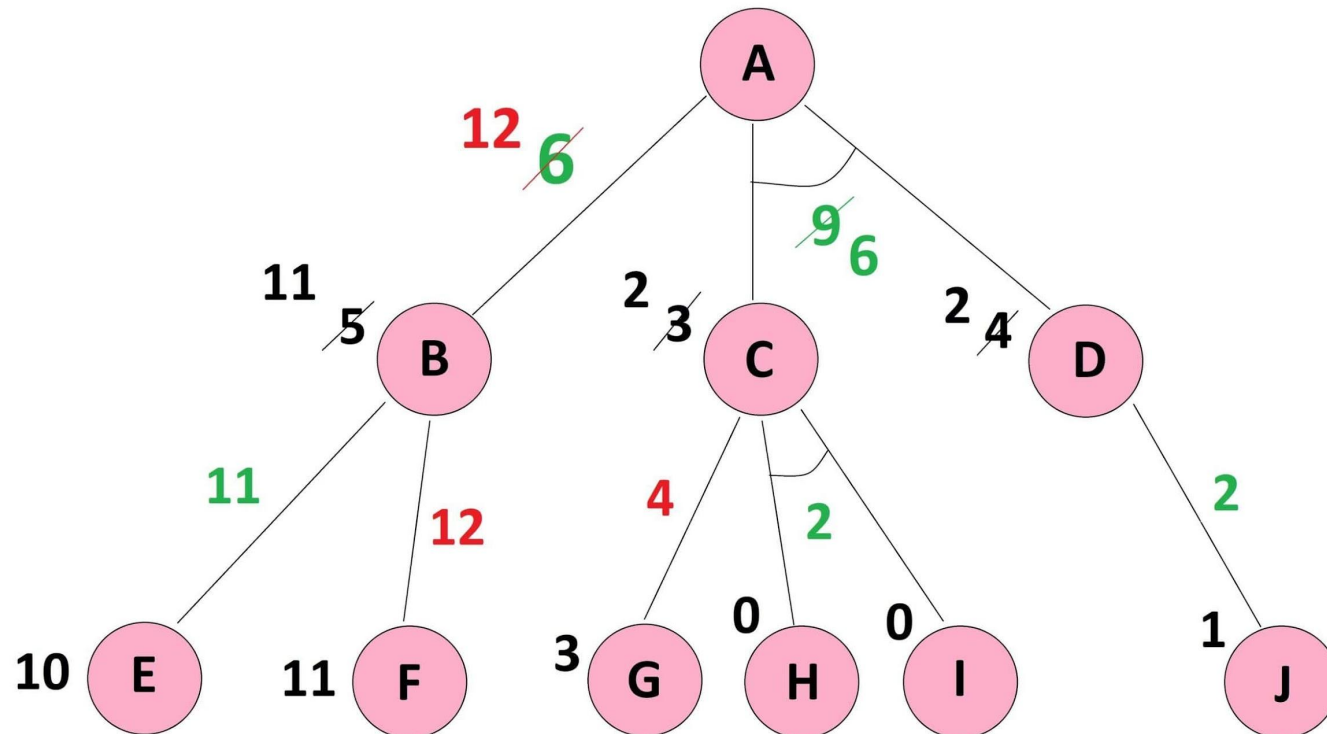
7. AO* (AND-OR-Star) Algorithm – example



7. AO* (AND-OR-Star) Algorithm – example



7. AO* (AND-OR-Star) Algorithm – example



8.Hill-Climbing Search

(1/6)

- A local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem.
- It terminates when it reaches a peak value where no neighbor has a higher value.

8. Hill-Climbing Search (2/6)

1) Hill - Climbing Search

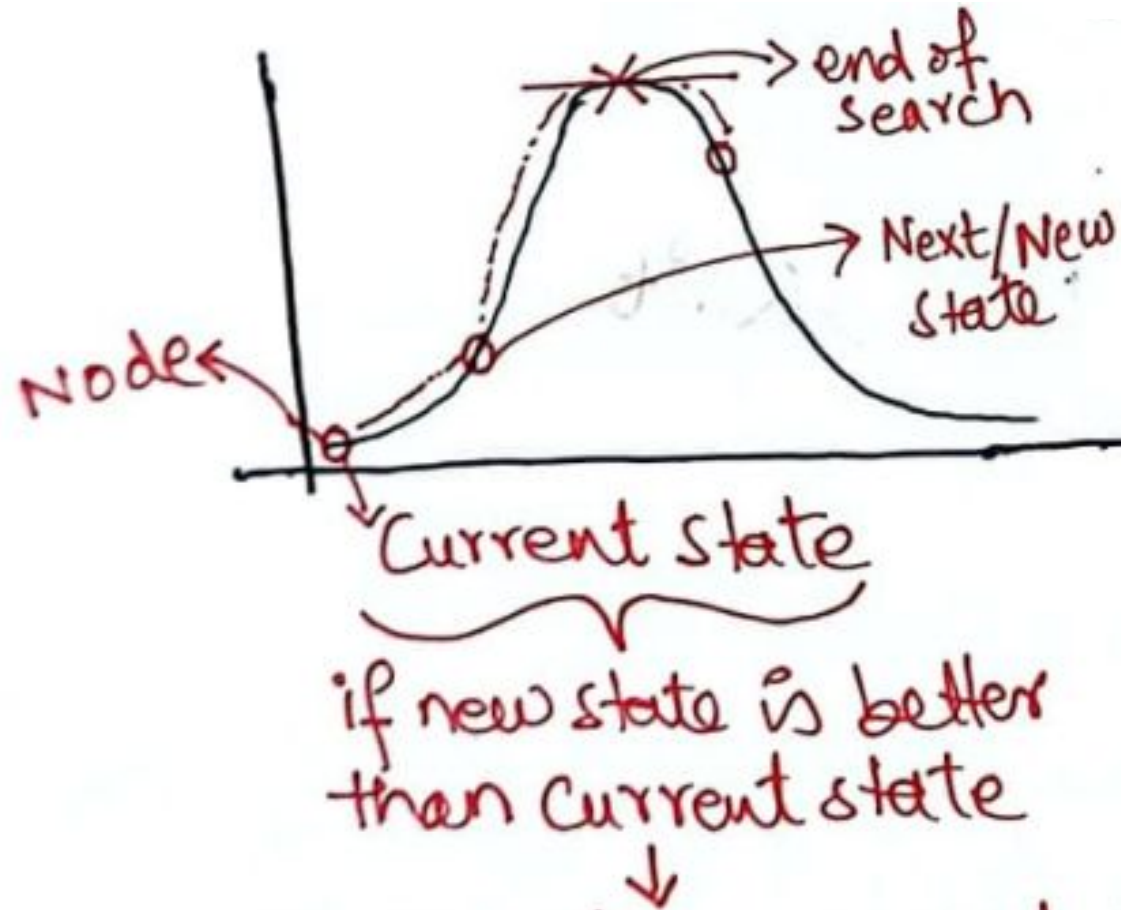
↳ Variant of generate and test method in which feedback from test procedure is used to help generator decide which direction to move in search space.] $\Rightarrow$ Always moves in a single direction.

↳ It is like DFS (Depth First Search).

↳ '3' Important points

- i - Local Search Algorithm (Local Maxima).
- ii - It uses Greedy Approach.
- iii - No Back Tracking.

8. Hill-Climbing Search (3/6)

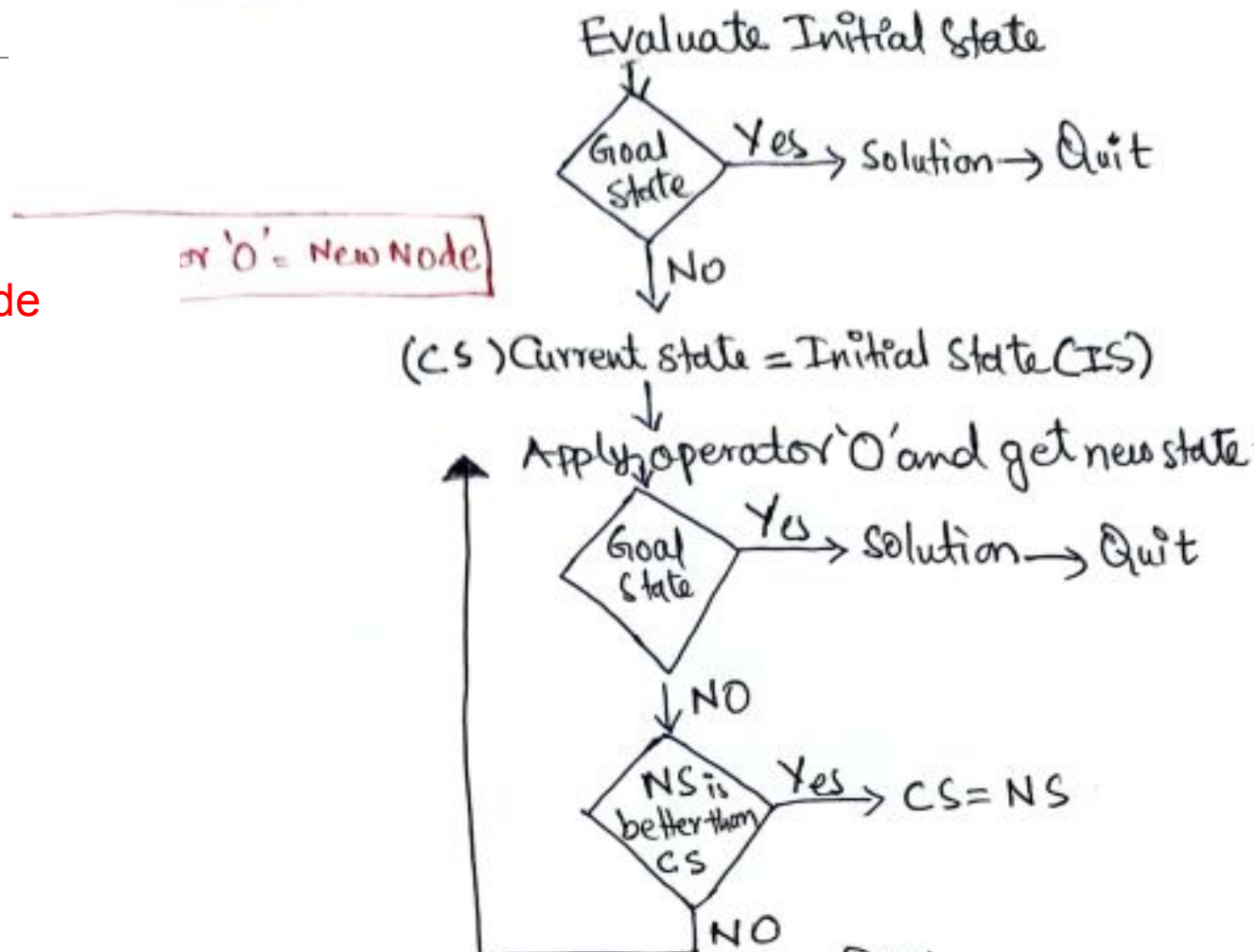


Current state = new state

8. Hill-Climbing Search (4/6)

Flowchart:

Operator 'O' = New Node



8. Hill-Climbing Search (5/6)

Example

1	2	4
5		7
3	6	8

⇒ Starting State/Initial State

⇒ Let's say the next three states have heuristic values as 4, 5 & 6. The state with minimum heuristic value will be selected as Next/New State.

(4) (5) ~~(6)~~

↳ We will choose '4' & discard '5' & '6'

8. Hill-Climbing Search (6/6)

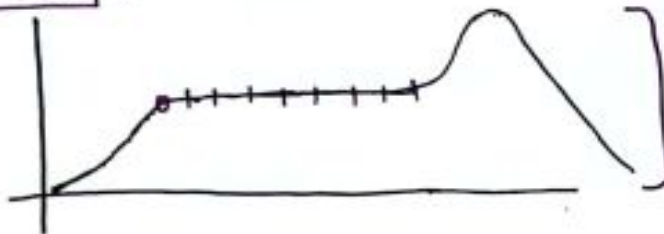
Limitations of Hill-Climbing Search:

① Local Maxima:



Complete Domain knowledge is not known.

② Plateau:



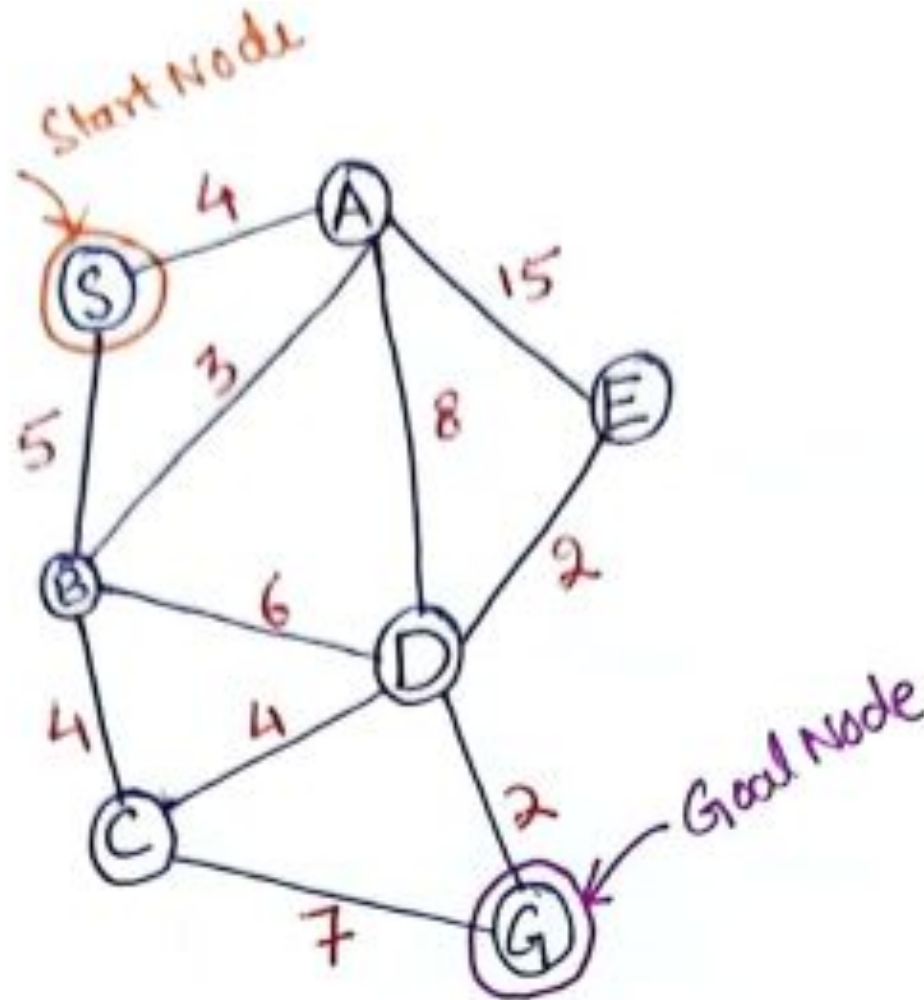
Algo gets same value / Same solⁿ for some time.

9. Branch and Bound Technique (1/4)

- ↳ Branch and Bound search is a way to combine space saving of depth first search with heuristic information.
- ↳ Optimal solution is chosen among many solution paths.
- ↳ Idea in Branch-and-Bound search is to maintain lowest-cost path to a GOAL found so far, and its cost.
- ↳ Typically used with depth-first search.
- ↳ No back tracking [i.e., only forward direction].
- ↳ Two parts
 - i- Branch: Several choices are found
 - ii- Bound: Setting Bound on solution Quality
 - ↳ Pruning: Trimming of branches where solution Quality is poor.

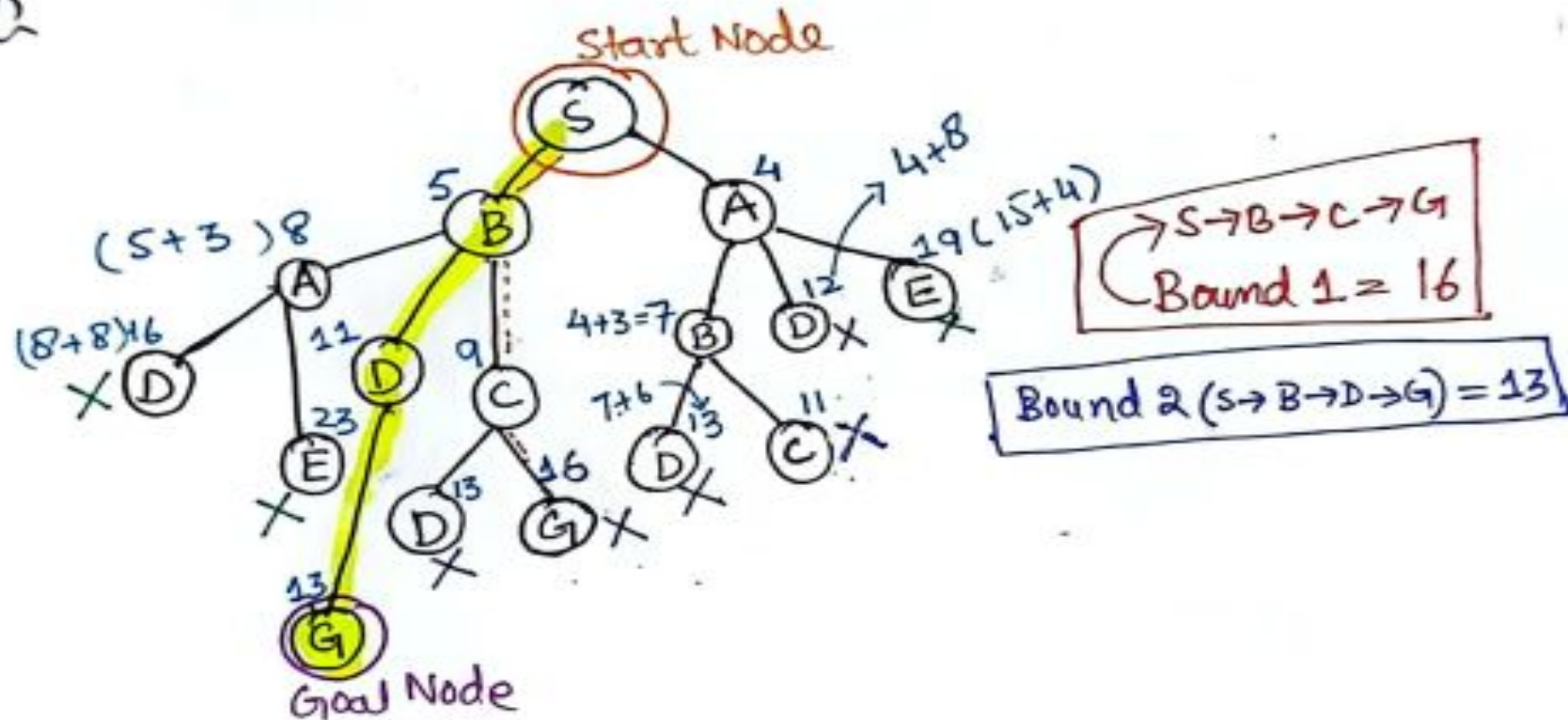
9.Branch and Bound Technique (2/4)

Example:

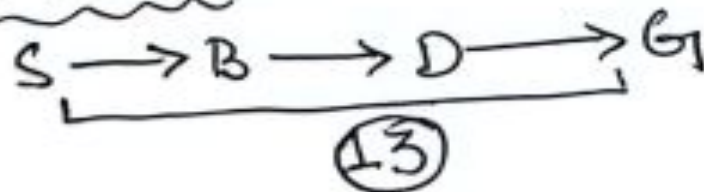


9.Branch and Bound Technique (3/4)

Solution



Optimal Path



9.Branch and Bound Technique (4/4)

Disadvantages:

- i- load balancing factor of branch and bound makes the parallelization difficult.
- ii- It is limited to small sized network.

10.State Space Search (1/5)

4) State Space Search:

- ↳ It is used in problem solving.
- ↳ It is a process used in A.I in which successive configurations or states of an instance are considered with intention of finding a GOAL state with desired property.
- ↳ Problems are modelled as State Space
set of states in which a problem can exist.

10.State Space Search (2/5)

↳ Representation:

$S: [S, A, \text{Action}(S), \text{Result}(s,a), \text{Cost}(s,a)]$

Where

- S = set of all possible states of a Problem
- A = set of all possible actions
- $\text{Action}(S)$ = It is a function which determines which action is possible for current state.
- $\text{Result}(s, a)$ = It is a function which tells about the next state reached by performing action 'a' on a current state 's'.
- $\text{Cost}(s, a)$ = It is a function which tells about the cost when action 'a' is performed on state 's'.

10.State Space Search (3/5)

Example 1 Eight-Tile Puzzle Problem

Start State (S)

1	4	3
2	///	5
8	6	7

GOAL STATE (G)

1	2	3
8	///	4
7	6	5

Possible Actions: 4  } Legal Moves for a particular state

↳ for example 'Right Move' on Start State (S) above :

1	4	3
2	5	///
8	6	7

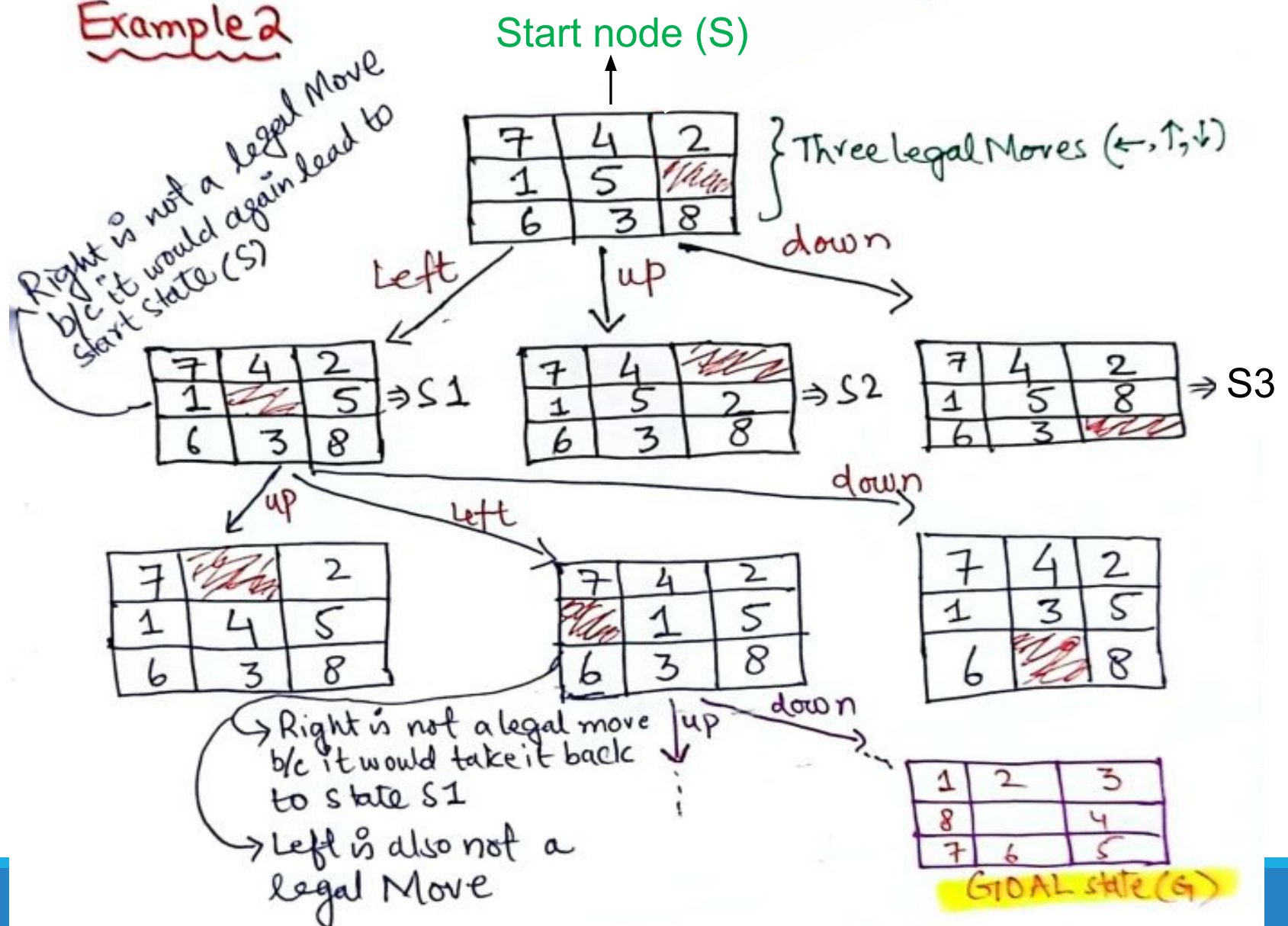
↓
New State (S1)

} Right is not possible now } illegal Move

 } Legal Moves

10.State Space Search (4/5)

Example 2

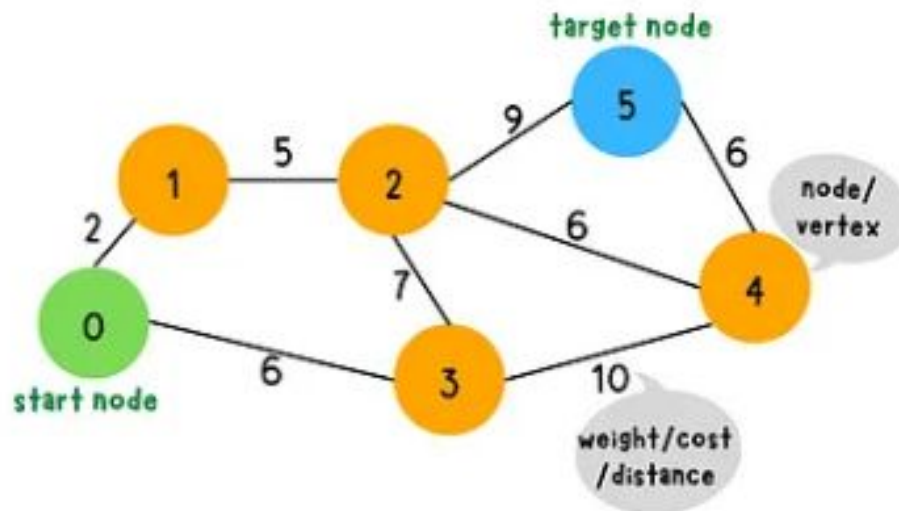


10.State Space Search (5/5)

Home Task:

Determine the minimum number of moves to reach to GOAL node (G) in Example 2.

Activity



$h(n)$

node	Heuristic distance to target node
0	20
1	16
2	6
3	10
4	4
5	0

References: Textbooks

1. *“Artificial Intelligence”, by Elaine Rich and Kevin Knight, (2006), McGraw Hill companies Inc., Chapter 1-22 page 1-613.*
2. *“Artificial Intelligence: A Modern Approach” by Stuart Russell and Peter Norvig, (2010), Prentice Hall, Chapter 1-27. page 1-1151.*
3. *“Computational Intelligence: A logical Approach”, by David Poole, Alan Mackworth, and Randy Goebel, (1998), Oxford University Press, Chapter 1-12, page 1-608.*
4. *“Artificial Intelligence: Structures and Strategies for Complex Problem Solving”, by George F.Lugar, (2002), Addison-Wesley, Chapter 1-16, page 1-743.*
5. *“AI: A new Synthesis”, by Nils J.Nilsson, (1998), Morgan Kaufmann Inc., Chapter 1-25, Page 1-493.*
6. *“Artificial Intelligence: Theory and Practice” by Thomas Dean, (1994), Addison-Wesley, Chapter 1-10, Page 1-650.*
7. *Related documents from open source, mainly internet.*